

Revisão de segurança do Medde

Data: 09/10/2026 · Escopo: código do repositório (app, funções do servidor, migrações do banco, rotinas do GitHub) e dependências.

Resumo em 5 linhas

- 1) A base está sólida: cada conta só lê os próprios dados, as chaves secretas ficam no servidor e não encontrei senhas no código.
- 2) Achei e **corrigi no código** 4 problemas (1 alto, 1 médio, 2 baixos).
- 3) Restam **2 itens altos que dependem de você**: trocar o token e a senha do banco, e ligar a verificação em duas etapas.
- 4) O maior risco de custo é abuso da IA com contas falsas: configure um limite de gasto na OpenAI.
- 5) Isto **não substitui** um teste de invasão profissional. Veja os limites no final.

1. O que foi corrigido no código

#	Gravidade	Problema	O que fiz
1	Alta	O Next.js 16.3.7 tinha 6 vulnerabilidades conhecidas (uma alta: requisição forçada pelo otimizador de imagens; quatro médias de cache que podiam misturar conteúdo entre usuários; uma baixa).	Atualizei para Next.js 16.3.8 . A verificação de dependências agora diz "nenhuma vulnerabilidade conhecida". Testes e build passam.
2	Média	Na função de notificações, qualquer endereço https era aceito como destino. Uma conta cadastrada poderia fazer o servidor enviar requisições a sites escolhidos por ela (SSRF cega).	Agora só são aceitos os serviços de push do Google, Mozilla, Apple e Windows. Endereços com usuário, senha ou porta também são recusados. Testes novos.
3	Baixa	O site não enviava cabeçalhos de segurança (proteção contra ser colocado dentro de outro site, contra tipo de arquivo trocado e contra vazamento do endereço de origem).	Adicionei 5 cabeçalhos. A política CSP (mais rígida) não foi ligada, porque precisa de testes próprios e pode quebrar o app.

4	Baixa	Na leitura de recibo, o tipo da imagem enviada pelo usuário ia sem validação para a OpenAI.	Agora só passa jpeg, png, webp e heic.
---	--------------	---	--

Estas correções estão no branch de desenvolvimento. Entram no ar quando você disser "pode publicar".

2. O que depende de você

#	Gravidade	Risco	O que fazer
5	Alta	Token do Supabase e senha do banco apareceram em conversa e prints. Quem tiver o token controla a sua conta Supabase; quem tiver a senha lê e apaga todos os dados.	Trocar os dois (Parte 1 do guia). 15 minutos.
6	Alta	Contas suas sem verificação em duas etapas (GitHub, Supabase, Vercel, Google, Hostinger, OpenAI). Roubar uma delas dá acesso a tudo.	Ativar a verificação em duas etapas em todas, com aplicativo autenticador. Guardar os códigos de recuperação.
7	Média	Abuso de custo da IA. O cadastro não exige confirmar o e-mail. Cada conta falsa pode usar até 40 áudios e 60 recibos por dia na sua chave da OpenAI.	Em platform.openai.com, defina um limite de gasto mensal (Billing → Limits). Antes de abrir ao público, ligue a confirmação de e-mail.
8	Média	Só o e-mail protege o painel do administrador. Hoje funciona porque a sua conta já existe.	No Supabase (Edge Functions → Secrets), crie <code>ADMIN_USER_ID</code> com o id da sua conta (Authentication → Users). Assim o painel só abre para esse id.
9	Média	Links do orçamento não expiram. Quem receber o link (ou a conversa do WhatsApp) vê nome, endereço e valores para sempre, a menos que o pintor cancele o link.	Decidir se quer validade (por exemplo 90 dias). Posso implementar.
10	Baixa	Um robô pode encher as 40 vagas diárias da página de pedidos de um pintor e impedir pedidos reais.	Futuro: adicionar um captcha (por exemplo Cloudflare Turnstile).

11	Baixa	Os dados guardados no celular não são criptografados. Quem pegar o celular desbloqueado vê os dados.	Orientar o uso de bloqueio de tela. O app apaga os dados ao sair da conta.
12	Baixa	A senha mínima é de 6 caracteres.	No Supabase, Authentication, aumente para 8 ou mais. Também no app (<code>AuthGate.tsx</code>).
13	Baixa	Não há alerta de acesso suspeito nem atualização automática de bibliotecas.	GitHub: Settings → Code security → ativar Dependabot alerts e security updates (grátis). Olhe os Logs das funções de vez em quando.

3. O que já estava bem

- **Dados por conta:** todas as tabelas têm RLS ligado. `user_data` , `subscriptions` , `shared_quotes` , `public_pages` , `quote_requests` e `push_subscriptions` só liberam a leitura da própria linha. Tabelas de uso (`voice_usage` , `receipt_usage`), `push_keys` e `admin_settings` não têm política: só as funções do servidor acessam.
- **Fotos e áudios:** bucket privado, cada pessoa acessa só a própria pasta (`visit-files`).
- **Assinatura:** o usuário só lê a própria linha; ninguém consegue se dar acesso pelo app.
- **Chaves:** as chaves secretas (OpenAI, administrador do Supabase) só existem no servidor. As variáveis públicas do app são só as públicas. Não encontrei chaves, tokens nem senhas no código ou no histórico.
- **Funções:** as que exigem login confirmam o usuário pelo token (e não por um id enviado). Excluir conta só exclui a própria. A função interna de aviso compara a chave em tempo constante. Entradas são limpas e limitadas (tamanho, formato, https).
- **Links:** o token do link tem 144 bits aleatórios (impossível de adivinhar). O link de pagamento só aceita https. O texto vindo de usuários é exibido pelo React sem HTML cru (não há `dangerouslySetInnerHTML` no código).
- **Página pública de pedidos:** campo-isca contra robôs, limite diário, limite de pedidos abertos, mesmo telefone não repete, aviso no celular só nos 5 primeiros do dia.
- **Rotinas do GitHub:** os nomes recebidos são validados, as permissões são mínimas, e a rotina de SQL nunca roda sozinha.

4. Confira você mesmo no banco (5 minutos)

Eu li os arquivos SQL do repositório, mas **não consigo ver o banco real**. Uma migração pode não ter sido rodada (a 0003 não tinha sido). Confira:

- 1 Abra [Supabase](#) → [SQL Editor](#), cole este comando e toque em **Run**:

```
select tablename, rowsecurity
from pg_tables where schemaname = 'public'
order by tablename;
```

Todas as linhas devem mostrar `rowsecurity = true`. Se alguma estiver `false`, me mande o print.

- 2 Depois rode este e mande o print do resultado:

```
select schemaname, tablename, policyname, cmd
from pg_policies
where schemaname in ('public', 'storage')
order by schemaname, tablename;
```

- 3 Confira em [Storage](#) que o bucket `visit-files` aparece como **Private** (não Public).

5. Limites desta revisão

- Foi uma **revisão de código** feita por IA, e não um teste de invasão (pentest). Não tentei atacar o app em produção.
- Não vi as configurações dos painéis do Supabase (Authentication, limites, SMTP), da Vercel e do GitHub, nem se as migrações estão todas rodadas.
- Não revisei pagamento (ainda não existe) nem o app da Play Store (ainda não existe).
- Falhas de lógica de negócio e abusos mais criativos podem passar. Antes de ter centenas de usuários ou pagamentos, contrate uma auditoria profissional.
- A segurança é um processo: repita esta revisão quando houver novidades grandes (pagamento, novas funções públicas).

Depois de cada mudança grande, peça

Faça uma revisão de segurança das mudanças: autenticação e autorização de cada função nova, RLS de tabelas novas, validação de entradas, segredos no código, dependências com vulnerabilidades, e dados pessoais expostos. Liste por gravidade e proponha correção.